

INT102 全面知识点汇总

整理依据：当前文件夹中的课件笔记、Algorithm 笔记、远伴期末参考资料、Exam Guide、Exam Solutions、2020–2023 Final/Resit 往年题、2024 CW1/CW2。

说明：部分往年卷和 suggested solution 是扫描版，普通文字提取不到；本汇总优先依据可完整读取的文本资料，并结合文件名中对应年份资料的题型信息整理。

1. 考试结构与高频规律

1.1 近年题型趋势

早期试卷常见结构：

- Part A: 选择题，通常 60 分。
- Part B: 大题，通常 40 分。

2022–2023 以后出现过：

- Part I: 28 道选择题，每题 2.5 分，共 70 分。
- Part II: 大题，共 30 分。

你现在要求的结构“28 选择 70 分，大题 30 分”与 2022–2023 的结构最接近。

1.2 高频考点排序

最高优先级：

1. Time Complexity / Big-O: 几乎每套卷都会考，选择题 3–4 道很常见。
2. BFS / DFS: 遍历顺序、邻接表/邻接矩阵、最后访问顶点。
3. MST: Prim、Kruskal、MST 是否唯一。
4. Dijkstra: 从源点出发的最短路标签、顶点选择顺序、shortest path tree 与 MST 区别。
5. Sorting: Bubble Sort、Selection Sort、Insertion Sort、Merge Sort，尤其比较次数和交换次数。
6. Divide and Conquer: 大题高频，常考 bitonic peak、min/max、merge sort recurrence。
7. Dynamic Programming: 0/1 Knapsack、Coin Row、Change-making、LCS、Assembly Line、Sequence Alignment。
8. P / NP / NP-Complete: 定义、归约、证明模板，4-SAT、CLIQUE、Independent Set 等。

中高优先级：

- Horspool 字符串匹配。
- Floyd-Warshall / Bellman-Ford。
- Graph representation: adjacency matrix/list、incidence matrix/list。
- Euler circuit/path、Hamiltonian path/circuit。
- Greedy algorithm 的适用与反例。

较低但不能完全放弃：

- Branch and Bound。
- Travelling Salesman Problem。
- Job Assignment。
- Warshall transitive closure。

2. 算法与伪代码基础

2.1 Algorithm 的定义

Algorithm 是解决问题的一组明确、有限、可执行的步骤。

好的算法通常满足：

- Input: 有清楚输入。
- Output: 有清楚输出。
- Definiteness: 每步明确无歧义。
- Finiteness: 有限步后停止。
- Effectiveness: 每步都能实际执行。

2.2 Algorithm 与 Program 的区别

- Algorithm 是思想和步骤，可以用伪代码描述。
- Program 是某种编程语言中的具体实现。
- INT102 更重视伪代码、复杂度、算法思想，不要求严格语法。

2.3 常见伪代码结构

顺序结构：

```
input n
sum = 0
output sum
```

条件结构：

```
if condition then
    statement
else
    statement
```

循环结构：

```
for i = 1 to n do
    statement

while condition do
    statement
```

递归结构：

```
Algorithm F(n)
    if n == 0 then
        return base value
    else
        return expression involving F(smaller input)
```

考试易错点：

- 伪代码不需要写成 Java/Python。
- 递归必须有 base case。
- 写大题时一定要说明 input、output、核心步骤和复杂度。

3. 时间复杂度与 Big-O

3.1 为什么分析复杂度

复杂度用于衡量算法随输入规模 n 增长时的效率。

两个主要指标：

- Time complexity: 关键操作执行次数。
- Space complexity: 额外内存使用量。

考试中常考：

- “efficiency of algorithm is measured by time and space”

- “time factor is measured by number of key operations”
- “space factor is measured by memory needed”

3.2 Big-O 正确定义

$f(n)$ is $O(g(n))$ 当且仅当存在常数 $c > 0$ 和 $n_0 > 0$, 使得对所有 $n \geq n_0$:

$$f(n) \leq c * g(n)$$

易错选项:

- “for all n , $f(n) \leq g(n)$ ” 不完整。
- “ $O(n(n+1)/2)$ ” 没有化简, 应写 $O(n^2)$ 。
- Big-O 描述上界, 不一定是精确增长率。

3.3 增长率排序

从慢到快:

$O(1)$
 $O(\log \log n)$
 $O(\log n)$
 $O(\sqrt{n})$
 $O(n)$
 $O(n \log n)$
 $O(n^2)$
 $O(n^3)$
 $O(2^n)$
 $O(n!)$

常见比较:

- $\log n$ 比 n 慢。
- $n \log n$ 比 n 快, 比 n^2 慢。
- 多项式通常比指数慢。
- $n!$ 比 2^n 增长更快。

3.4 常见代码复杂度

```
sum = n * (n + 1) / 2
```

复杂度: $O(1)$ 。

陷阱: 虽然公式里有 n , 但只做常数运算。

```
for i = 1 to n:
    constant work
```

复杂度: $O(n)$ 。

```
for i = 1 to n:
    for j = 1 to n:
        constant work
```

复杂度: $O(n^2)$ 。

```
x = 1
while x < n:
    x = x * 2
```

复杂度: $O(\log n)$ 。

原因: x 每次翻倍。

```
for i = 1 to n:
    for j = 1 to i:
        constant work
```

复杂度: $O(n^2)$ 。

原因: $1 + 2 + \dots + n = n(n+1)/2$ 。

3.5 常见递推式

Binary Search:

$$T(n) = T(n/2) + 1$$
$$T(n) = O(\log n)$$

Merge Sort:

$$T(n) = 2T(n/2) + n$$
$$T(n) = O(n \log n)$$

Divide and Conquer 找最大/最小:

$$T(n) = 2T(n/2) + O(1)$$
$$T(n) = O(n)$$

Bitonic peak:

$$T(n) = T(n/2) + O(1)$$
$$T(n) = O(\log n)$$

指数递归:

$$T(n) = 2T(n-1) + 1$$
$$T(n) = O(2^n)$$

3.6 Big-O 证明模板

若要证明:

$$f(n) = 9n + 3n^2 + 2n \log n + 3 \text{ is } O(n^2)$$

思路:

对 $n \geq 2$, 有:

$$n \leq n^2$$
$$n \log n \leq n^2$$
$$3 \leq 3n^2$$

所以:

$$f(n) \leq 9n^2 + 3n^2 + 2n^2 + 3n^2 = 17n^2$$

取 $c = 17$, $n_0 = 2$, 即可证明 $f(n)$ is $O(n^2)$ 。

4. 搜索算法

4.1 Linear Search

思想: 从头到尾逐个比较。

伪代码:

```
LinearSearch(A[0..n-1], x)
  for i = 0 to n-1 do
    if A[i] == x then
      return i
  return -1
```

复杂度:

- Best case: $O(1)$, 目标在第一个位置。
- Worst case: $O(n)$, 目标不存在或在最后。
- Average case: $O(n)$ 。

常考点:

- worst case occurs when item is not in the array.
- best case occurs when item is first element.

4.2 Binary Search

前提：数组已排序。

思想：每次比较中间元素，排除一半搜索空间。

伪代码：

```
BinarySearch(A[0..n-1], x)
  l = 0
  r = n - 1
  while l <= r do
    m = floor((l + r) / 2)
    if A[m] == x then
      return m
    else if A[m] < x then
      l = m + 1
    else
      r = m - 1
  return -1
```

复杂度： $O(\log n)$ 。

易错点：

- 未排序数组不能直接二分。
- 递推式是 $T(n) = T(n/2) + O(1)$ 。

5. 排序算法

5.1 Bubble Sort

思想：相邻元素逆序就交换，一轮后最大或最小元素归位。

常见伪代码：

```
BubbleSort(A[0..n-1])
  for i = 0 to n - 2 do
    for j = n - 1 downto i + 1 do
      if A[j] < A[j - 1] then
        swap A[j] and A[j - 1]
```

比较次数：

$$(n-1) + (n-2) + \dots + 1 = n(n-1)/2$$

复杂度：

- Worst/Average: $O(n^2)$ 。
- 如果没有 early stopping, best case 比较次数仍为 $O(n^2)$ 。

交换次数：

- 等于数组中的 inversion 数量。
- 已排序数组交换 0 次。
- 逆序数组交换 $n(n-1)/2$ 次。

5.2 Selection Sort

思想：每轮从未排序部分选择最小元素放到前面。

比较次数：

$$(n-1) + (n-2) + \dots + 1 = n(n-1)/2$$

复杂度： $O(n^2)$ 。

交换次数：最多 $n - 1$ 次，取决于实现和输入。

考试常考：

- 长度 6 数组比较次数： $5 + 4 + 3 + 2 + 1 = 15$ 。
- Selection Sort 比较次数与初始顺序无关。

5.3 Insertion Sort

思想：把当前元素插入前面已排序部分。

复杂度：

- Best case: $O(n)$ ，数组已排序。
- Worst case: $O(n^2)$ ，数组逆序。

5.4 Merge Sort

思想：分治。

步骤：

1. Divide：把数组分成两半。
2. Conquer：递归排序左右两半。
3. Combine：合并两个有序数组。

递推式：

$$T(n) = 2T(n/2) + n$$
$$T(1) = 1$$

复杂度：

$$T(n) = O(n \log n)$$

大题常考：

- 解释递推式中 $2T(n/2)$ 和 n 的含义。
- 用 substitution 或 iteration 证明 $O(n \log n)$ 。

5.5 Counting Sort

适用：键值范围较小的整数排序。

复杂度：

$$O(n + k)$$

其中 k 是取值范围大小。

6. Divide and Conquer 分治法

6.1 核心思想

Divide and Conquer 包含三步：

1. Divide：把原问题划分成规模更小的子问题。
2. Conquer：递归解决子问题。
3. Combine：合并子问题答案得到原问题答案。

适合场景：

- 子问题结构相同。
- 子问题相互独立。
- 合并成本可控。

6.2 与 Dynamic Programming 的区别

Divide and Conquer:

- 子问题通常独立。
- 不一定保存子问题结果。
- 例子: Merge Sort、Binary Search、Bitonic Peak。

Dynamic Programming:

- 子问题重叠。
- 保存子问题结果, 避免重复计算。
- 例子: Knapsack、LCS、Coin Row。

6.3 Bitonic Peak Finding

题型: 数组先严格递增后严格递减, 找 peak 位置 p。

暴力法:

```
for i = 1 to n:
    if A[i-1] < A[i] and A[i] > A[i+1]:
        return i
```

复杂度: $O(n)$ 。

分治法思路:

- 取 mid。
- 如果 $A[mid] < A[mid + 1]$, peak 在右边。
- 否则 peak 在左边或 mid。

伪代码:

```
Peak(A, l, r)
    if l == r then
        return l
    mid = floor((l + r) / 2)
    if A[mid] < A[mid + 1] then
        return Peak(A, mid + 1, r)
    else
        return Peak(A, l, mid)
```

递推式:

$$T(n) = T(n/2) + O(1)$$

复杂度: $O(\log n)$ 。

真题特点:

- 2021-2022 Final Part B 考过 bitonic peak。
- 2022-2023 Resit Part II 也考过类似题。

6.4 MinMax 分治

题型: 给定 n 个数, 同时找最大值和最小值。

伪代码:

```
MinMax(A, l, r)
    if l == r then
        return (A[l], A[l])
    if r == l + 1 then
        if A[l] <= A[r] then
            return (A[l], A[r])
        else
            return (A[r], A[l])

    mid = floor((l + r) / 2)
```

```

(min1, max1) = MinMax(A, l, mid)
(min2, max2) = MinMax(A, mid + 1, r)
return (min(min1, min2), max(max1, max2))

```

递推式:

$$T(n) = 2T(n/2) + 2$$

复杂度: $O(n)$ 。

真题特点:

- 2020-2021 Resit Part B 考过。

6.5 分治大题答题模板

写大题时按这个顺序:

1. State idea: 说明 divide、conquer、combine。
2. Pseudocode: 写清楚 base case 和 recursive case。
3. Recurrence: 写 $T(n)$ 并解释每一项。
4. Complexity: 解递推式。
5. Correctness: 说明左右子问题正确, 合并正确。

7. Graph Theory 图论基础

7.1 图的基本概念

Graph $G = (V, E)$:

- V : vertices 顶点集合。
- E : edges 边集合。

类型:

- Undirected graph: 无向图。
- Directed graph: 有向图。
- Weighted graph: 加权图。
- Connected graph: 连通图。

7.2 度数 Degree

无向图中:

- $\text{degree}(v)$: 与 v 相连的边数。
- 总度数 = $2 \times$ 边数。

有向图中:

- indegree: 入度。
- outdegree: 出度。

7.3 图的表示

Adjacency Matrix:

- $n \times n$ 矩阵。
- 若 i 和 j 有边, 则 $A[i][j] = 1$ 或 weight。
- 空间 $O(n^2)$ 。
- 查询是否有边快。

Adjacency List:

- 每个顶点存邻居列表。

- 空间 $O(V + E)$ 。
- 稀疏图更适合。

Incidence Matrix:

- 行表示顶点，列表示边。
- 若顶点与边相连，则填 1。

Incidence List:

- 每条边列出它连接的两个顶点。

CW1 考过:

- adjacency matrix。
- adjacency list。
- incidence matrix。
- incidence list。

7.4 Path、Circuit、Cycle

Path: 顶点序列，相邻顶点之间有边。

Circuit: 起点和终点相同的 path。

Cycle: 除起点终点外不重复的 circuit。

7.5 Euler Path / Euler Circuit

Euler path: 经过每条边一次。

Euler circuit: 经过每条边一次并回到起点。

无向连通图判断:

- Euler circuit 存在当且仅当所有顶点度数为偶数。
- Euler path 存在当且仅当恰好 0 个或 2 个顶点度数为奇数。

易错点:

- Euler 看边是否每条走一次。
- Hamiltonian 看顶点是否每个走一次。

7.6 Hamiltonian Path / Circuit

Hamiltonian path: 经过每个顶点一次。

Hamiltonian circuit: 经过每个顶点一次并回到起点。

通常比 Euler 更难判断，和 NP-Complete 内容有关。

8. BFS 与 DFS

8.1 BFS Breadth-First Search

思想: 按层访问，使用 queue。

伪代码:

```
BFS(G, s)
    mark s as visited
    enqueue s
    while queue is not empty do
        v = dequeue
        visit v
        for each unvisited neighbor u of v do
            mark u as visited
            enqueue u
```

复杂度:

$O(V + E)$

常考:

- 给邻接表, 从某点开始写 BFS 顺序。
- 若有多个邻居, 按题目规定顺序, 常见是 alphabetical order。

8.2 DFS Depth-First Search

思想: 一路走到底, 再回溯。可用 recursion 或 stack。

伪代码:

```
DFS(G, v)
    mark v as visited
    visit v
    for each unvisited neighbor u of v do
        DFS(G, u)
```

复杂度:

$O(V + E)$

常考:

- DFS 第 5 个访问的顶点。
- DFS 最后一个访问的顶点。
- DFS tree。

8.3 BFS/DFS 易错点

- 标记 visited 的时机要一致。考试通常按“发现时标记”。
- 邻居访问顺序非常重要。
- BFS 使用 queue, DFS 使用 stack/recursion。
- BFS 在无权图中可求最短边数路径。

9. Greedy Methods 贪心算法

9.1 贪心思想

每一步选择当前看来最好的选项, 希望得到全局最优。

适用需要满足:

- Greedy choice property: 局部最优选择可导向全局最优。
- Optimal substructure: 最优解包含子问题最优解。

9.2 Change-making 贪心

标准币值如 25、10、5、1 时, 贪心通常有效:

```
GreedyChange(n, d[1..m])
    i = m
    while n > 0 do
        if d[i] <= n then
            output d[i]
            n = n - d[i]
        else
            i = i - 1
```

贪心反例:

```
coins = {1, 4, 5}, amount = 8
Greedy: 5 + 1 + 1 + 1 = 4 coins
Optimal: 4 + 4 = 2 coins
```

真题特点:

- 2020-2021 Final Part B 考过 Change-making: 贪心、反例、DP 递推、伪代码和复杂度。

9.3 MST 中的贪心

Prim 和 Kruskal 都是贪心算法。

贪心点:

- Prim: 每次从当前树向外选最小权边。
- Kruskal: 每次选全图当前最小且不成环的边。

10. Minimum Spanning Tree

10.1 MST 定义

Minimum Spanning Tree:

- 覆盖所有顶点。
- 是一棵树, 所以有 $|V| - 1$ 条边。
- 总权重最小。

适用: 连通、无向、加权图。

10.2 Kruskal Algorithm

步骤:

1. 按边权从小到大排序。
2. 依次尝试加入边。
3. 若加入后不形成 cycle, 则保留。
4. 直到选出 $|V| - 1$ 条边。

伪代码:

```
Kruskal(G)
  T = empty set
  sort edges by increasing weight
  for each edge e in sorted order do
    if e does not create a cycle in T then
      add e to T
  return T
```

常考:

- 写 selected edge order。
- 判断某条边是否在 MST。
- 判断 MST 是否唯一。

10.3 Prim Algorithm

步骤:

1. 从指定起点开始。
2. 每次选择连接“已在树中顶点”和“未在树中顶点”的最小权边。
3. 加入新顶点。
4. 重复直到所有顶点加入。

伪代码:

```
Prim(G, s)
  T = empty set
  S = {s}
  while S does not contain all vertices do
```

```

    choose minimum-weight edge (u, v)
    where u in S and v not in S
    add (u, v) to T
    add v to S
return T

```

常考：

- 从指定起点写 vertex order。
- 写 priority queue 的变化。
- 与 Dijkstra 对比。

10.4 MST 唯一性

若所有边权互不相同，则 MST 一定唯一。

若有相同边权，MST 可能不唯一，但不一定不唯一。

11. Shortest Path Algorithms

11.1 Dijkstra Algorithm

适用：

- 单源最短路径。
- 边权非负。

思想：

- 维护从源点到每个顶点的当前最短估计 dist。
- 每次选择 dist 最小且未确定的顶点。
- 用该顶点松弛邻边。

伪代码：

```

Dijkstra(G, s)
  for each vertex v do
    dist[v] = infinity
    prev[v] = null
  dist[s] = 0
  Q = all vertices
  while Q is not empty do
    u = vertex in Q with minimum dist[u]
    remove u from Q
    for each neighbor v of u do
      if dist[u] + w(u, v) < dist[v] then
        dist[v] = dist[u] + w(u, v)
        prev[v] = u

```

常考：

- 每一步 label 更新。
- 顶点确定顺序。
- shortest path tree。

易错点：

- Dijkstra 不能保证处理负权边正确。
- Dijkstra tree 不一定是 MST。

11.2 Bellman-Ford Algorithm

适用：

- 单源最短路径。
- 可以处理负权边。

- 可以检测负权环。

步骤:

1. 初始化 $\text{dist}[s] = 0$, 其他 infinity 。
2. 重复 $|V| - 1$ 轮松弛所有边。
3. 再检查一轮, 如果还能松弛, 则存在负权环。

复杂度:

$O(VE)$

CW2 考过 Bellman-Ford。

11.3 Floyd-Warshall Algorithm

用途: 所有点对最短路径。

状态:

$D[k][i][j] = \text{从 } i \text{ 到 } j \text{ 经过中间点 } k \text{ 的最短距离}$

递推:

$D[k][i][j] = \min(D[k-1][i][j], D[k-1][i][k] + D[k-1][k][j])$

复杂度:

$O(n^3)$

常考:

- 给 k 的范围, 求某两个顶点当前最短距离。
- 与 Dijkstra 区分: Floyd 是 all-pairs shortest paths。

11.4 Warshall Algorithm

用途: transitive closure, 判断可达性。

递推:

$R[k][i][j] = R[k-1][i][j] \text{ OR } (R[k-1][i][k] \text{ AND } R[k-1][k][j])$

12. Dynamic Programming 动态规划

12.1 DP 核心思想

Dynamic Programming 适合具有:

- Optimal substructure: 最优解由子问题最优解组成。
- Overlapping subproblems: 子问题重复出现。

步骤:

1. 定义状态。
2. 写递推式。
3. 写初始条件。
4. 按正确顺序填表。
5. 从表中读出答案, 必要时回溯解。
6. 写复杂度。

12.2 Coin Row Problem

问题: 一排硬币 $c_1..c_n$, 不能选择相邻硬币, 最大化金额。

状态:

$F[i] = \text{max}(F[i-1], c[i] + F[i-2])$

递推:

$F[0] = 0$
 $F[1] = c[1]$
 $F[i] = \max(F[i-1], c[i] + F[i-2])$

解释:

- 不选第 i 个: $F[i-1]$ 。
- 选第 i 个: $c[i] + F[i-2]$ 。

复杂度:

Time: $O(n)$
Space: $O(n)$

真题特点:

- 2022-2023 Final Part II 考过 Coin Row。

12.3 Change-making DP

问题: 用不限数量硬币组成金额 n , 最少硬币数。

状态:

$F[x] = \text{make amount } x$

递推:

$F[0] = 0$
 $F[x] = \min(F[x - d[i]] + 1) \text{ for all } d[i] \leq x$

伪代码:

```
DPChange(n, d[1..m])
  F[0] = 0
  for x = 1 to n do
    F[x] = infinity
    for i = 1 to m do
      if d[i] <= x then
        F[x] = min(F[x], F[x - d[i]] + 1)
  return F[n]
```

复杂度:

$O(nm)$

12.4 0/1 Knapsack

问题: 每个物品最多选一次, 在容量 W 内最大化价值。

状态:

$V[i, w] = \text{max value with } i \text{ items and weight } w$

初始条件:

$V[0, w] = 0$
 $V[i, 0] = 0$

递推:

```
if weight[i] > w:
  V[i, w] = V[i - 1, w]
else:
  V[i, w] = max(
    V[i - 1, w],
    value[i] + V[i - 1, w - weight[i]]
  )
```

复杂度:

Time: $O(nw)$

Space: $O(nw)$ ■■■■■■ $O(w)$

回溯选物品:

从 $V[n, W]$ 开始:

- 若 $V[i, w] == V[i - 1, w]$, 说明 item i 未选。
- 否则 item i 被选, $w = w - \text{weight}[i]$, $i = i - 1$ 。

真题特点:

- 选择题常考表中某个 $V[i, w]$ 。
- 大题常要求填表、最大价值、最优物品集合。
- 2022-2023 Resit Part II 考过 0/1 Knapsack。

12.5 LCS Longest Common Subsequence

问题: 找两个字符串最长公共子序列长度。

状态:

$c[i, j] = X[1..i] \text{ ■ } Y[1..j] \text{ ■ LCS} \text{ ■■}$

初始条件:

$c[0, j] = 0$

$c[i, 0] = 0$

递推:

```
if  $X[i] == Y[j]$ :  
     $c[i, j] = c[i - 1, j - 1] + 1$   
else:  
     $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$ 
```

复杂度:

$O(mn)$

常考:

- 填 DP 表。
- 问某个 $c[i, j]$ 。
- 给出一个 LCS。

易错点:

- Subsequence 不要求连续。
- Substring 才要求连续。

12.6 Assembly Line Scheduling

问题: 两条装配线, 每条有 n 个 station, 可在 station 间换线, 求最短总时间。

状态:

$f1[j] = \text{■■ line 1 ■ station } j \text{ ■■■■■}$

$f2[j] = \text{■■ line 2 ■ station } j \text{ ■■■■■}$

递推:

```
 $f1[j] = \min(f1[j-1] + a1[j], f2[j-1] + t2[j-1] + a1[j])$   
 $f2[j] = \min(f2[j-1] + a2[j], f1[j-1] + t1[j-1] + a2[j])$ 
```

还要记录 line information, 方便回溯 fastest way。

CW2 考过:

- 填 $f_i[j]$ 表。
- 求 f^* 。
- 根据 line information 找 fastest path。

12.7 Sequence Alignment

CW2 重点: global alignment 和 local alignment。

Global Alignment: Needleman-Wunsch

用途: 全局序列比对, 整段序列都参与比对。

初始化:

```
F[0, 0] = 0
F[i, 0] = i * gap
F[0, j] = j * gap
```

递推:

```
F[i, j] = max(
    F[i - 1, j - 1] + score(x_i, y_j),
    F[i - 1, j] + gap,
    F[i, j - 1] + gap
)
```

答案: $F[m, n]$ 。

Local Alignment: Smith-Waterman

用途: 找两个序列中最相似的局部片段。

初始化:

```
F[0, j] = 0
F[i, 0] = 0
```

递推:

```
F[i, j] = max(
    0,
    F[i - 1, j - 1] + score(x_i, y_j),
    F[i - 1, j] + gap,
    F[i, j - 1] + gap
)
```

答案: 表中最大值。

区别:

- Global 从右下角回溯。
- Local 从全表最大值回溯, 到 0 停止。

13. String Matching 字符串匹配

13.1 Brute-force String Matching

思想: 文本中每个位置都尝试匹配模式串。

复杂度:

$O(nm)$

13.2 Horspool Algorithm

核心: 从模式串右端开始比较, 失配后根据 text 中对齐的字符查 shift table。

Shift table 构造:

模式串长度为 m 。

默认:

$\text{shift}[c] = m$

对模式串前 $m - 1$ 个字符:

$\text{shift}[\text{pattern}[i]] = m - 1 - i$

注意:

- 最后一个字符不参与表的更新。
- 若字符重复, 以最右边出现的位置为准。

应用步骤:

1. 将 pattern 与 text 某位置对齐。
2. 从 pattern 最右字符开始向左比较。
3. 若全部匹配, 则找到。
4. 若失配, 根据 text 中当前与 pattern 最后一位对齐的字符移动。

CW2 考过:

- 给 pattern CGTGC, 为 A/G/C/T 建 shift table。
- 在 text AGCCGTGC 中应用 Horspool, 计算比较次数。

14. P、NP、NP-Complete

14.1 Decision Problem

Decision problem 的答案是 yes/no。

复杂性理论里 P、NP、NPC 通常讨论 decision problem。

14.2 Class P

P: 可以在多项式时间内求解的问题集合。

例子:

- Sorting。
- Shortest path with nonnegative weights。
- MST。

为什么 P 被认为 efficient:

- 多项式时间通常被视为可有效计算。

14.3 Class NP

NP: 给定一个 certificate, 可以在多项式时间内验证 yes answer 的问题集合。

重点:

- NP 不代表 “not polynomial”。
- NP 是 nondeterministic polynomial time。

例子:

- SAT。
- CLIQUE。
- Hamiltonian Cycle。
- Subset Sum。

14.4 NP-Complete

问题 X 是 NP-Complete, 当且仅当:

1. X belongs to NP.
2. Every problem in NP can be polynomial-time reduced to X .

常用证明方式:

1. 证明 X in NP.
2. 选择一个已知 NP-Complete 问题 Y .
3. 构造 polynomial-time reduction $Y \leq_p X$.
4. 说明 Y yes iff X yes.
5. 得出 X is NP-Complete.

14.5 Polynomial-time Reduction

$A \leq_p B$ 表示:

- 存在多项式时间算法, 把 A 的任意实例转换成 B 的实例。
- A 的答案为 yes 当且仅当转换后的 B 实例答案为 yes。

用于证明 NP-hard:

- 若已知 A 是 NP-Complete, 且 $A \leq_p B$, 则 B 至少和 A 一样难, 所以 B 是 NP-hard.
- 若 B 还属于 NP, 则 B 是 NP-Complete.

14.6 4-SAT 证明思路

真题考过 4-SAT 是 NP-Complete。

答题结构:

1. 4-SAT in NP: 给定变量赋值, 可以多项式时间检查每个 clause 是否为 true。
2. 从 3-SAT 归约到 4-SAT。
3. 把每个 3-literal clause 转换成若干 4-literal clauses, 使可满足性保持等价。
4. 转换规模是多项式。
5. 因为 3-SAT 是 NP-Complete, 所以 4-SAT 是 NP-hard; 结合 in NP, 得到 NP-Complete。

14.7 CLIQUE 与 Independent Set

CLIQUE:

- 给定图 G 和 k , 问是否存在大小至少为 k 的 clique。

CLIQUE in NP:

- certificate 是 k 个顶点。
- 检查这 k 个顶点两两之间是否有边, $O(k^2)$ 。

Independent Set 到 CLIQUE:

- 给定 G , 构造补图 G' 。
- G 中大小 k 的 independent set 等价于 G' 中大小 k 的 clique。
- 构造补图是多项式时间。

结论:

- Independent Set $\leq_p$ CLIQUE。
- CLIQUE in NP。
- 因此 CLIQUE 是 NP-Complete。

14.8 P vs NP 常考判断

正确说法:

- P subset of NP。
- 若任一 NP-Complete 问题有多项式时间算法，则 $P = NP$ 。
- 到目前为止，一般课程默认 $P = NP$ 未被证明。

错误说法：

- NP means not polynomial。
- NP-Complete 一定没有多项式算法。
- NP 中所有问题都已知不能多项式求解。

15. Branch and Bound

15.1 核心理念

Branch and Bound 用于优化问题。

- Branch: 把搜索空间分成子问题。
- Bound: 计算上下界，剪掉不可能优于当前最优解的分支。

常见应用：

- Travelling Salesman Problem。
- Job Assignment Problem。
- Knapsack optimization。

15.2 与 Backtracking 的区别

Backtracking:

- 用 feasibility 剪枝。
- 常用于约束满足问题。

Branch and Bound:

- 用 objective bound 剪枝。
- 常用于求最优解。

16. 典型真题大题模板

16.1 Change-making 大题模板

可能问：

- 写 greedy algorithm。
- 给 greedy 失败反例。
- 写 DP recurrence。
- 写 DP pseudocode 和 complexity。

答案骨架：

Greedy: always choose the largest coin not exceeding the remaining amount.
 Counterexample: coins {1,4,5}, amount 8.
 DP state: $F[x]$ = minimum number of coins for amount x .
 Recurrence: $F[x] = \min(F[x-d_i] + 1)$.
 Complexity: $O(nm)$.

16.2 Divide and Conquer 大题模板

可能问：

- 解释思想。

- 设计算法。
- 写 recurrence。
- 用 induction/iteration 证明复杂度。

答案骨架：

Divide the problem into smaller subproblems.
Solve subproblems recursively.
Combine their solutions.
Base case handles input of size 1.
 $T(n) = \dots$
Therefore $T(n) = \dots$
Correctness follows by induction on input size.

16.3 DP 填表题模板

必须写：

1. 状态定义。
2. 初始条件。
3. 递推式。
4. 表格。
5. 最优值。
6. 回溯得到选择方案。
7. 时间和空间复杂度。

16.4 NP-Complete 证明模板

To prove X is NP-complete:

1. X is in NP because a certificate can be verified in polynomial time.
2. Choose a known NP-complete problem Y.
3. Construct a polynomial-time reduction from Y to X.
4. Prove yes-instance equivalence.
5. Therefore X is NP-hard.
6. Since X is in NP, X is NP-complete.

17. 选择题速记表

必背结论		
存在 $c > 0$ 和 n_0 , 使 $f(n) \leq c g(n)$		
公式直接计算通常是常数时间		
$O(\log n)$		
worst $O(n)$, best $O(1)$		
$n(n-1)/2$		
$n(n-1)/2$		
$T(n) = 2T(n/2) + n$, $O(n \log n)$		
$T(n) = T(n/2) + 1$, $O(\log n)$		
queue, $O(V+E)$		
stack/recursion, $O(V+E)$		
连通无向图所有顶点度数为偶数		
\	$V \setminus$	-1
按边权从小到大, 不成环就选		
从起点扩张, 每次选跨 cut 最小边		
非负边权, 单源最短路		

	必背结论		
	可处理负权边，可检测负环		
	all-pairs shortest paths, $O(n^3)$		
	$V[i, w]$ 前 i 个物品容量 w 最大价值		
	不要求连续		
	shift table 决定失配移动		
	多项式时间可求解		
	多项式时间可验证		
	in NP 且 NP-hard		

18. 考前复习优先级

第一优先级：必须会

- Big-O 定义、增长率排序、代码复杂度。
- 递推式： $T(n)=T(n/2)+1$, $T(n)=2T(n/2)+n$, $T(n)=2T(n/2)+1$ 。
- BFS/DFS 按给定顺序遍历。
- Prim、Kruskal、Dijkstra 手算。
- Bubble/Selection 比较次数和交换次数。
- Divide and Conquer 大题模板。
- 0/1 Knapsack 填表。
- Coin Row recurrence。
- P/NP/NPC 定义和 reduction 模板。

第二优先级：很可能考

- LCS 填表和回溯。
- Horspool shift table。
- Change-making greedy 反例和 DP。
- Assembly Line Scheduling。
- Sequence Alignment: global/local。
- Floyd-Warshall 某一步矩阵。
- Bellman-Ford 松弛。

第三优先级：保底补充

- Incidence matrix/list。
- Euler/Hamiltonian 区别。
- Branch and Bound。
- TSP / Job Assignment 基本概念。
- Warshall transitive closure。

19. 常见失分点

1. Big-O 不化简，例如写 $O(n(n+1)/2)$ 。
2. 把 $O(\log n)$ 和 $O(n \log n)$ 顺序排错。
3. BFS/DFS 没按题目给的邻居顺序。
4. Kruskal 选边时忘记检查 cycle。

5. Prim 和 Dijkstra 混淆: Prim 看边权, Dijkstra 看从源点累计距离。
6. Dijkstra 用在负权边上。
7. 0/1 Knapsack 写成 unbounded knapsack。
8. LCS 当成 substring。
9. DP 大题只填答案不写状态和递推式。
10. NP-Complete 证明方向写反: 应从已知 NPC 问题归约到目标问题。
11. 只证明 NP-hard, 忘记证明 in NP。
12. Sequence local alignment 忘记和 0 比较。

20. 一页背诵版

Big-O:

$f(n)=O(g(n))$ iff exists $c>0, n_0>0$ such that $f(n)\leq c*g(n)$ for all $n\geq n_0$.

Growth:

$1 < \log\log n < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < 2^n < n!$

Search:

Linear $O(n)$, Binary $O(\log n)$.

Sort:

Bubble/Selection $O(n^2)$, comparisons $n(n-1)/2$.

Merge Sort $T(n)=2T(n/2)+n=O(n\log n)$.

Graph:

BFS queue, DFS recursion/stack, both $O(V+E)$.

Euler circuit iff all degrees even.

MST has $|V|-1$ edges.

Kruskal sort edges; Prim grow from start.

Dijkstra nonnegative single-source shortest paths.

Bellman-Ford allows negative edges.

Floyd all-pairs shortest paths $O(n^3)$.

DP:

Coin row $F[i]=\max(F[i-1], c[i]+F[i-2])$.

Knapsack $V[i,w]=\max(V[i-1,w], v_i+V[i-1,w-w_i])$.

LCS if equal +1 else $\max(\text{top}, \text{left})$.

Global alignment initializes gap penalties.

Local alignment includes max with 0.

Complexity:

P = polynomial-time solvable.

NP = polynomial-time verifiable.

NPC = in NP and every NP problem reduces to it.

To prove NPC: in NP + known NPC $\leq_p$ target.